

Part 1 : Function

```
In [1]: print("L")
```

L

```
In [11]: def addie(x , y):  
         print(f"{x} & {y} are being added")
```

```
In [12]: addie(3,6)
```

3 & 6 are being added

```
In [3]: def add2(x, y):  
        print(f"{x} & {y} are being added")  
        pass
```

```
In [10]: add2(30,20)
```

30 & 20 are being added

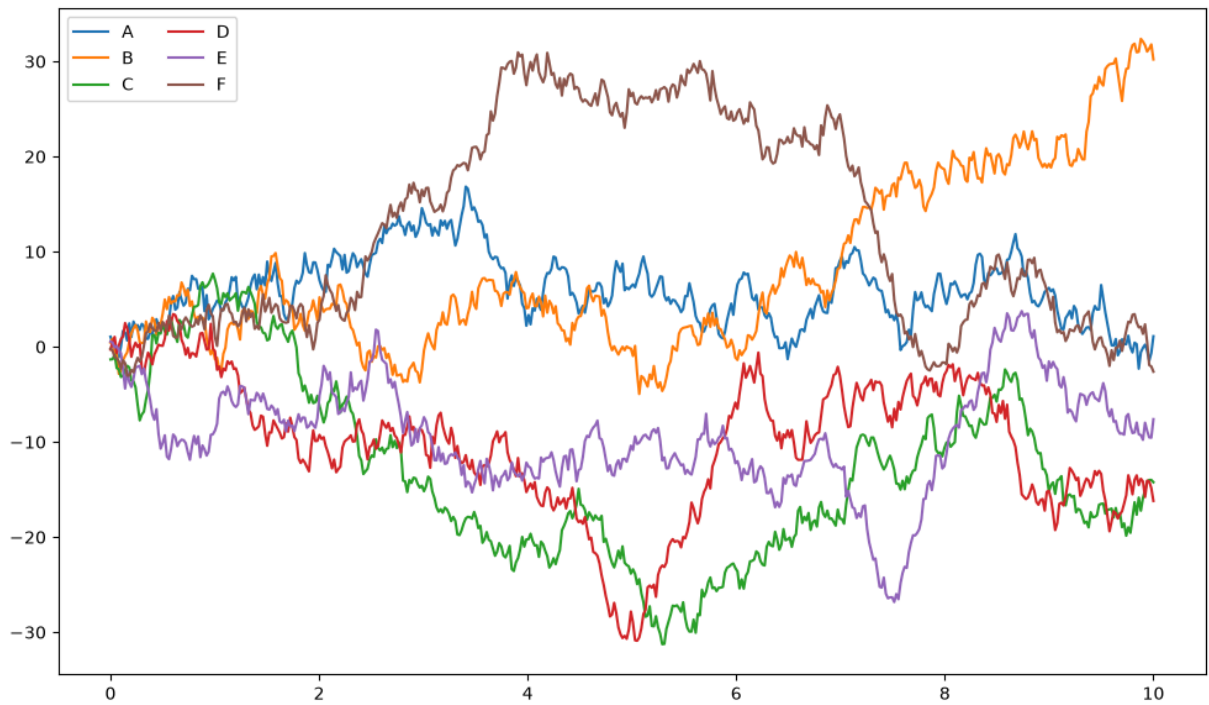
Part 2 : Matplotlib

```
In [37]: import numpy as np  
         import matplotlib.pyplot as plt  
         %matplotlib inline
```

```
In [12]: x = np.linspace(0, 10, 500)  
         y = np.cumsum(np.random.randn(500, 6), 0)
```

```
In [13]: plt.figure(figsize=(12, 7))  
         plt.plot(x, y)  
         plt.legend('ABCDEF', ncol=2, loc='upper left')
```

```
Out[13]: <matplotlib.legend.Legend at 0x188fce06cf0>
```



Part 2 : API request

```
In [2]: import requests
import pandas as pd
```

requesting data from API kraken

```
In [3]: def get_historic_price(symbol, after='2018-09-01'):

    url = 'https://api.kraken.com/0/public/OHLC'
    pair = f"{symbol.upper()}USD" # XBTUSD when symbol='xbt' for example

    resp = requests.get(url, params={
        "pair": pair,
        'interval': 60,
        'since': str(int(pd.Timestamp(after).timestamp()))
    })
    resp.raise_for_status()

    data = resp.json()

    results_key = [k for k in data['result'].keys() if k != 'last'][0]
    results = [
        (close_time, float(open), float(high), float(low), float(close), float(volume))
        for (close_time, open, high, low, close, vwap, volume, count)
        in data['result'][results_key]
    ]
    df = pd.DataFrame(results, columns=[
        'CloseTime', 'OpenPrice', 'HighPrice', 'LowPrice', 'ClosePrice', 'Volume'
    ])
    df['CloseTime'] = pd.to_datetime(df['CloseTime'], unit='s')
```

```
df.set_index('CloseTime', inplace=True)  
return df
```

This function fetches historical hourly OHLC (Open, High, Low, Close) price data for a specified cryptocurrency from the Kraken public API, processes the response, and formats it into a clean, time-indexed pandas DataFrame ready for analysis.

Function Parameters

symbol: The ticker string for the cryptocurrency asset (e.g., 'xbt' or 'eth'), which is converted to uppercase and combined with USD to form Kraken's trading pair format (e.g., XBTUSD).

after: A string representing the starting date (defaulting to '2018-09-01') from which to pull historical data.

API Request Mechanics

It sends an HTTP GET request to Kraken's public endpoint (<https://api.kraken.com/0/public/OHLC>).

It sets the interval to 60, which requests 1-hour candlestick data.

It converts the after date string into a Unix timestamp using `pd.Timestamp(after).timestamp()` and passes it to the since parameter so Kraken only returns data forward from that date.

`resp.raise_for_status()` ensures the script halts and throws an error if the API request fails (e.g., a 404 or 500 error).

Data Parsing and Cleansing

`resp.json()` parses the incoming JSON payload.

A list comprehension isolates the correct data array by finding the key in `data['result']` that isn't the 'last' pagination marker.

It unpacks Kraken's raw 8-item candle arrays—keeping only the close time, open, high, low, close, and volume, while casting them all to floats. VWAP and trade count are intentionally dropped.

DataFrame Formatting

It builds a pandas DataFrame assigning descriptive column names: CloseTime, OpenPrice, HighPrice, LowPrice, ClosePrice, and Volume.

It transforms the Unix epoch integer in CloseTime into a readable UTC datetime object using `pd.to_datetime(..., unit='s')`.

It sets CloseTime as the DataFrame index, making it ideal for time-series operations, filtering, and plotting inside Jupyter.

```
In [4]: last_week = (pd.Timestamp.now() - pd.offsets.Day(7))
last_week
```

Out[4]: Timestamp('2026-08-23 19:28:47.415685')

```
In [5]: btc = get_historic_price('btc', after=last_week)
```

```
In [6]: eth = get_historic_price('eth', after=last_week)
```

Bitcoin

```
In [7]: btc.head()
```

Out[7]:

	OpenPrice	HighPrice	LowPrice	ClosePrice	Volume
CloseTime					
2026-08-23 20:00:00	77333.1	77449.8	77199.0	77397.6	22.974304
2026-08-23 21:00:00	77397.7	78035.3	77397.6	77834.0	454.185830
2026-08-23 22:00:00	77833.8	77990.2	77578.1	77601.8	64.158592
2026-08-23 23:00:00	77600.1	77801.6	77315.2	77737.5	159.532509
2026-08-24 00:00:00	77737.3	77737.4	77161.3	77642.9	60.102556

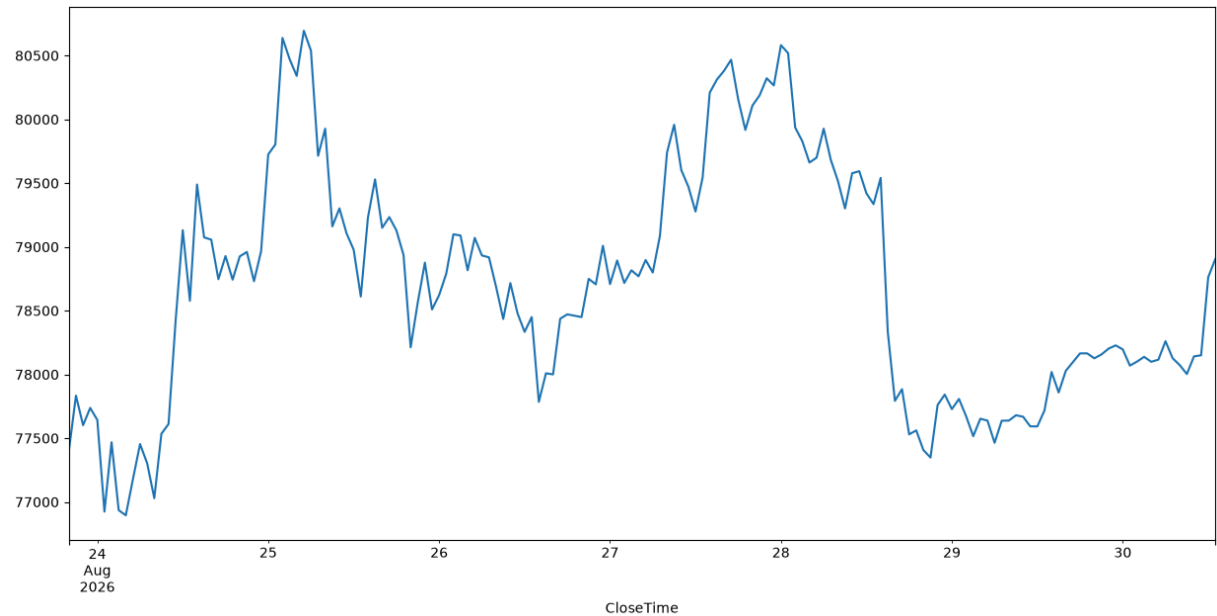
```
In [8]: btc.info()

<class 'pandas.DataFrame'>
DatetimeIndex: 162 entries, 2026-08-23 20:00:00 to 2026-08-30 13:00:00
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   OpenPrice    162 non-null    float64
1   HighPrice    162 non-null    float64
2   LowPrice     162 non-null    float64
3   ClosePrice   162 non-null    float64
4   Volume       162 non-null    float64
dtypes: float64(5)
memory usage: 7.6 KB
```

Part 3 : Graphing

```
In [9]: btc['ClosePrice'].plot(figsize=(15, 7))
```

Out[9]: <Axes: xlabel='CloseTime'>



```
In [10]: btc.head()
```

Out[10]:

	OpenPrice	HighPrice	LowPrice	ClosePrice	Volume
2026-08-23 20:00:00	77333.1	77449.8	77199.0	77397.6	22.974304
2026-08-23 21:00:00	77397.7	78035.3	77397.6	77834.0	454.185830
2026-08-23 22:00:00	77833.8	77990.2	77578.1	77601.8	64.158592
2026-08-23 23:00:00	77600.1	77801.6	77315.2	77737.5	159.532509
2026-08-24 00:00:00	77737.3	77737.4	77161.3	77642.9	60.102556

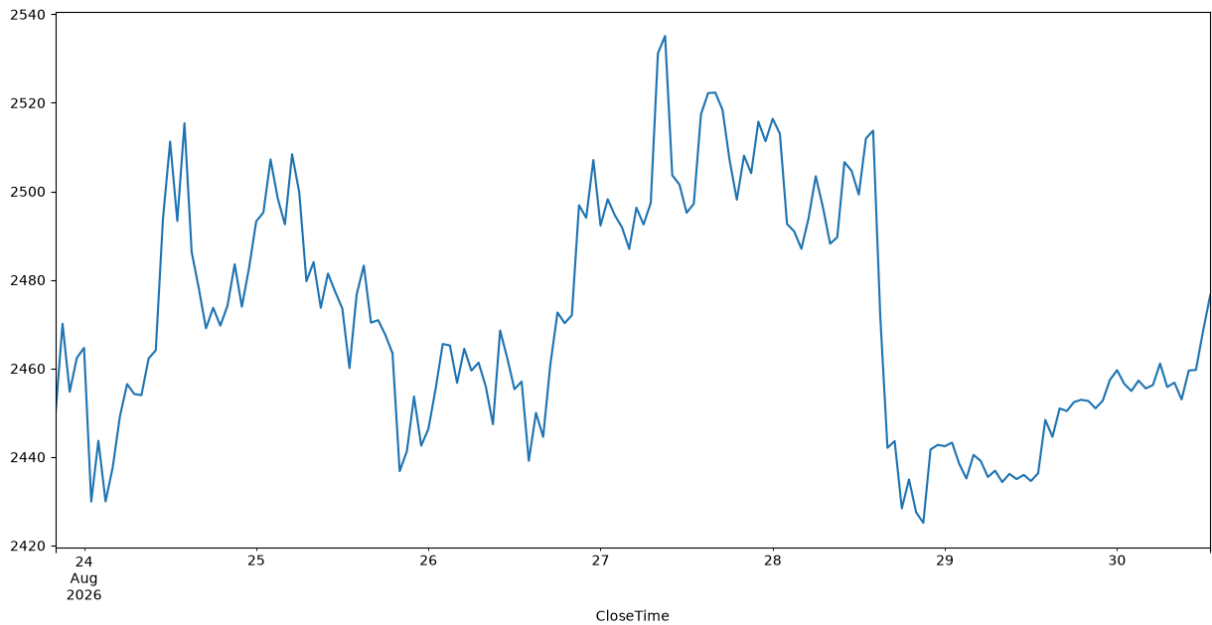
Ether

```
In [34]: eth.info()
```

```
<class 'pandas.DataFrame'>
DatetimeIndex: 162 entries, 2026-08-23 20:00:00 to 2026-08-30 13:00:00
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   OpenPrice   162 non-null    float64
1   HighPrice   162 non-null    float64
2   LowPrice    162 non-null    float64
3   ClosePrice  162 non-null    float64
4   Volume      162 non-null    float64
dtypes: float64(5)
memory usage: 7.6 KB
```

```
In [12]: eth['ClosePrice'].plot(figsize=(15, 7))
```

Out[12]: <Axes: xlabel='CloseTime'>



Bonus: Dynamic plots with Bokeh

```
In [13]: from bokeh.plotting import figure, output_file, show
         from bokeh.io import output_notebook
```

```
In [14]: output_notebook()
```



Loading BokehJS ...

```
In [15]: p1 = figure(x_axis_type="datetime", title="Crypto Prices", width=800)
         p1.grid.grid_line_alpha=0.3
         p1.xaxis.axis_label = 'Date'
         p1.yaxis.axis_label = 'Price'

         p1.line(btc.index, btc['ClosePrice'], color='#f2a900', legend_label='Bitcoin')
         #p1.line(eth.index, eth['ClosePrice'], color='#A6CEE3', legend_label='Ether')

         p1.legend.location = "top_left"

         show(p1)
```

`p1 = figure(x_axis_type="datetime", title="Crypto Prices", width=800)`: Initializes a new Bokeh plot canvas. It sets the X-axis to natively handle time-series data (like pandas DatetimeIndex), gives the plot the title "Crypto Prices", and sets the canvas width to 800 pixels.

`p1.grid.grid_line_alpha = 0.3`: Adjusts the opacity (alpha transparency) of the background grid lines to 0.3, making them subtle rather than distracting.

`p1.xaxis.axis_label = 'Date' & p1.yaxis.axis_label = 'Price'`: Assigns text labels to the horizontal and vertical axes.

`p1.line(btc.index, btc['ClosePrice'], color='#f2a900', legend_label='Bitcoin')`: Draws a line chart on the plot. It uses the `btc` DataFrame index (timestamps) for the X-axis and the `ClosePrice` column for the Y-axis. It colors the line Bitcoin gold (`#f2a900`) and adds a label for the legend.

`p1.line(eth.index, eth['ClosePrice'], color='#A6CEE3', legend_label='Ether')`: A commented-out line of code that would plot Ethereum prices in light blue if uncommented.

`p1.legend.location = "top_left"`: Moves the interactive legend box to the top-left corner of the chart.

`show(p1)`: Renders and displays the completed interactive chart inside your Jupyter Lab environment.

Part 4: Exporting to Excel

We'll now write both our Bitcoin and Ether data as separate sheets:

```
In [70]: import pandas as pd

# Write all sheets inside a single writer block using engine="openpyxl"
with pd.ExcelWriter("cryptos.xlsx", engine="openpyxl") as writer:
    btc.to_excel(writer, sheet_name="Bitcoin")
    eth.to_excel(writer, sheet_name="Etherr")
```

```
In [71]: # Read the sheet back into a DataFrame and display the first 5 rows
df = pd.read_excel("cryptos.xlsx", sheet_name="Bitcoin")
df.head()
```

```
Out[71]:
```

	CloseTime	OpenPrice	HighPrice	LowPrice	ClosePrice	Volume
0	2026-08-23 20:00:00	77333.1	77449.8	77199.0	77397.6	22.974304
1	2026-08-23 21:00:00	77397.7	78035.3	77397.6	77834.0	454.185830
2	2026-08-23 22:00:00	77833.8	77990.2	77578.1	77601.8	64.158592
3	2026-08-23 23:00:00	77600.1	77801.6	77315.2	77737.5	159.532509
4	2026-08-24 00:00:00	77737.3	77737.4	77161.3	77642.9	60.102556

```
In [72]: # Read the sheet back into a DataFrame and display the first 5 rows
df = pd.read_excel("cryptos.xlsx", sheet_name="Etherr")
df.head()
```

Out[72]:

	CloseTime	OpenPrice	HighPrice	LowPrice	ClosePrice	Volume
0	2026-08-23 20:00:00	2445.39	2450.29	2436.39	2449.25	1961.355945
1	2026-08-23 21:00:00	2449.26	2484.00	2449.26	2470.09	3213.715904
2	2026-08-23 22:00:00	2470.04	2471.93	2452.40	2454.74	811.397249
3	2026-08-23 23:00:00	2454.74	2470.00	2441.91	2462.40	922.208785
4	2026-08-24 00:00:00	2462.86	2464.65	2442.88	2464.65	786.464023

In []:

In []: